

Project Assignment Brief: Robotics Sensing, Actuation, ROS & Kinematics

Students must choose and complete **one (1)** of the three projects described in this document.

Course Domain:	Robotics Engineering
Topics Assessed:	Sensing, Actuation, ROS, Forward/Inverse Kinematics
Submission Mode:	Technical Report + Source Code + Live Demo (or Video)
Team Size:	Individual or Teams of up to 3 Students
Academic Year:	2025 / 2026

Project A — Robotic Arm

Project B — Line-Following
Robot

Project C — Landmark
Navigation

1. Overview and Learning Objectives

Purpose. This project assignment forms a core assessment component of the Robotics Engineering course. Each project is designed to evaluate the student's integrated understanding of four fundamental pillars of modern robotics: *sensing* (perception of the environment through sensors), *actuation* (command and control of motors and effectors), *Robot Operating System (ROS)* (middleware, node architecture, and inter-process communication), and *kinematics* (geometric modeling of robot motion, including forward and inverse kinematics). Students must select exactly one project and produce all required deliverables as specified below.

1.1 Core Learning Outcomes

Upon successful completion of the selected project, students will be able to:

1. Design and implement a sensor pipeline that perceives and interprets real-world signals (visual, proximity, inertial, or otherwise).
2. Select and drive appropriate actuators (servo motors, DC motors, thrusters, or end-effectors) to produce desired robot motions.
3. Structure a functional ROS workspace with nodes, topics, services, and launch files in accordance with ROS best practices.
4. Derive and apply kinematic models (forward and/or inverse) relevant to the robot morphology chosen.
5. Integrate sensing, actuation, ROS communication, and kinematic control into a cohesive, demonstrable robotic system.
6. Document engineering decisions, failure analyses, and experimental results in a formal technical report.

1.2 How to Read This Document

Each project description (Sections 3, 4, and 5) follows the same structure: background and motivation, technical requirements, ROS architecture expectations, kinematic expectations, deliverables checklist, and evaluation rubric. **Section 6 contains the unified marking rubric** applicable to all three projects. Section 7 specifies report format and submission instructions.

Important: Students must commit to one project by the stated deadline. Switching projects after the commitment deadline is not permitted unless approved in writing by the instructor.

2. General Technical Requirements (All Projects)

The following requirements apply regardless of which project is selected:

ROS Version

Students must use **ROS 2 (Humble Hawksbill or later)** as the primary middleware framework. Use of ROS 1 (Noetic) is permitted only with documented instructor approval and must include a rationale for the exception. All nodes, topics, and services must comply with ROS 2 architectural conventions.

Programming Language

Python 3 or C++17 (or a combination of both) within the ROS 2 package framework. Code must be version-controlled in a Git repository submitted as part of the deliverables.

Hardware or Simulation

Students may implement using physical hardware *or* a robotics simulator (Gazebo, Webots, or CoppeliaSim). If simulation is used, the simulation environment must faithfully reproduce the sensor modalities and actuator dynamics specified for the chosen project. Mixed implementations (hardware sensing + simulated actuators, or vice versa) must be declared explicitly.

Safety

Any physical hardware demonstration must include an observable emergency-stop (e-stop) mechanism that halts all actuators within 200 ms of activation. Documentation of the e-stop design is mandatory in the technical report.

Reproducibility

The submitted code repository must include a README.md with step-by-step instructions sufficient for the grader to build, launch, and reproduce the demonstration without contacting the student.

3. Project A — Open-Loop Robotic Arm Grasping

Project A

Open-Loop Robotic Arm Grasping

Design and control a multi-degree-of-freedom robotic arm to perform a structured grasping task using open-loop position control derived from forward kinematics.

3.1 Background and Motivation

Robotic manipulation is one of the oldest and most commercially important areas of robotics, underpinning industrial automation, surgical assistance, and warehouse logistics. In an open-loop control architecture, the robot executes a pre-computed sequence of joint commands without feedback correction. While limited in robustness, open-loop control provides the clearest foundation for understanding kinematics and actuation, because any error in joint angles manifests directly as end-effector positioning error, making kinematic modeling accuracy critical.

In this project, students design a robotic arm with a minimum of **three revolute joints**, compute its forward kinematic model, define a target grasp pose in Cartesian space, solve for the required joint

angles (analytically or numerically), and command the actuators via ROS 2 to achieve the grasp. No sensor feedback corrects the motion; accuracy depends entirely on the quality of the kinematic model and actuator calibration.

3.2 Sensor Requirements

Although the control loop is open, students must integrate at minimum **two sensing modalities**:

- **Joint position sensing:** Encoders or potentiometers on each joint to verify commanded vs. actual position after motion completion (used for error measurement, not feedback).
- **Object detection sensor:** A fixed proximity sensor (infrared, ultrasonic, or simple force sensor on the gripper) to confirm successful grasp — i.e., detect contact or closure. Camera-based object detection is acceptable and encouraged as an extension.

Sensor data must be published to ROS 2 topics and logged to a ROS 2 bag for post-analysis.

3.3 Actuation Requirements

- Minimum of **three actuated revolute joints** plus a gripper (open/close) as the end-effector.
- Each joint must be driven by a servo or stepper motor with documented torque and angular range specifications.
- Joint velocity and acceleration profiles must be defined (trapezoidal or polynomial trajectory planning) to avoid mechanical shock.
- Gripper must exert sufficient force to lift and hold the target object (specify object mass in the report).

3.4 ROS Architecture Requirements

Nodes	arm_controller	Publishes joint angle commands to actuators via the actuation driver node.
Nodes	sensor_reader	Reads joint encoders and proximity sensor; publishes to respective topics.
Nodes	kinematics_solver	Accepts a target Cartesian pose (service call) and returns joint

		angles via IK.
Nodes	grasp_sequencer	Orchestrates the reach, descend, grasp, and retract sequence via action or FSM.
Topics	/joint_states	Actual joint positions from encoders (sensor_msgs/JointState).
Topics	/arm_command	Target joint positions from arm_controller (sensor_msgs/JointState).
Topics	/grasp_contact	Boolean contact signal from gripper sensor (std_msgs/Bool).
Services	/solve_ik	Request: target pose (geometry_msgs/Pose); Response: joint angles (float64[]).
Launch File	arm_demo.launch.py	Launches all nodes, sets parameters, and starts ROS bag recording.

3.5 Kinematics Requirements

- **Denavit-Hartenberg (DH) parameterization:** Establish the DH table for the arm with all link lengths and joint offsets measured from the physical robot.
- **Forward Kinematics (FK):** Derive and implement the homogeneous transformation matrix chain T_{o_n} for n joints. Validate numerically against measured end-effector position.
- **Inverse Kinematics (IK):** Solve analytically (geometric or algebraic decoupling) or numerically (Jacobian pseudoinverse or optimization-based) for the target grasp pose. Document the solution method, singularity analysis, and workspace limits.
- **Trajectory planning:** Generate smooth joint-space trajectories using at minimum a cubic polynomial or trapezoidal velocity profile between the home configuration and the grasp configuration.

3.6 Demonstration Scenario

The arm begins at a defined home position. A static target object (e.g., a small cylinder or cube) is placed at a **known Cartesian coordinate** within the arm's reachable workspace. The system computes the IK solution, executes the reach and grasp trajectory open-loop, lifts the object at least 5 cm, and returns to the home position while holding the object. The demonstration must be repeatable three consecutive times.

3.7 Deliverables

- Technical report (format: Section 7) including DH table, FK derivation, IK derivation, trajectory design, and experimental error analysis.
 - ROS 2 package (full source) with `arm_demo.launch.py` and documented `README.md`.
 - ROS 2 bag file recording a complete successful grasp sequence.
 - Video recording (3–5 minutes) showing three consecutive successful grasps and narrated code walkthrough.
 - Error analysis table: commanded vs. measured joint angles; commanded vs. measured end-effector position in X, Y, Z.
-

4. Project B — Line-Following Wheeled Robot

Project B

Line-Following Wheeled Robot

Build and program a differential-drive wheeled robot to autonomously follow a marked line path using reactive sensing and a kinematic unicycle model.

4.1 Background and Motivation

Line-following is a classical benchmark in mobile robotics and is a practical foundation for autonomous guided vehicles (AGVs) used in manufacturing and logistics. Despite its apparent simplicity, achieving smooth, high-speed, and robust line-following requires careful sensor placement, signal processing, and control tuning — all of which reward a rigorous understanding of sensing and actuation.

In this project, students build a **differential-drive (two-wheel) robot** equipped with a line sensor array, drive it using wheel velocity commands computed by a proportional (or PID) controller, and model the robot's motion using the kinematic unicycle model (or differential-drive model with wheel radius and baseline as parameters). The robot must navigate a closed-loop line track that includes at least two 90-degree turns.

4.2 Sensor Requirements

- **Line sensor array:** A minimum of **5 infrared (IR) reflectance sensors** arranged in a row perpendicular to the robot's forward direction, capable of distinguishing the line (dark) from the ground (light) surface. A calibration routine must be implemented and documented.
- **Wheel encoders:** Incremental encoders on both drive wheels to measure actual wheel rotations. Encoder resolution must be stated in pulses per revolution (PPR).
- **Optional (encouraged):** An IMU (Inertial Measurement Unit) for heading estimation and odometry fusion.

All sensor values must be published as ROS 2 topics at a minimum rate of 20 Hz.

4.3 Actuation Requirements

- Two independently driven DC motors (one per wheel) with motor drivers (e.g., L298N or similar H-bridge).
- Motor commands expressed as geometry_msgs/Twist (linear velocity v , angular velocity ω) and converted to individual wheel velocities via the differential-drive kinematic model.
- PWM duty cycle limits must be defined to prevent motor stall or hardware damage.
- An audible or visual indicator (LED or buzzer) must signal completion of one full lap.

4.4 ROS Architecture Requirements

Nodes	line_sensor_node	Reads IR array; publishes weighted centroid error on

		/line_error.
Nodes	encoder_odometry_node	Reads wheel encoders; computes and publishes odometry on /odom.
Nodes	line_controller_node	Subscribes to /line_error; publishes geometry_msgs/Twist to /cmd_vel.
Nodes	motor_driver_node	Subscribes to /cmd_vel; converts to left/right wheel PWM signals.
Topics	/line_error	Normalized lateral error from line center (std_msgs/Float32).
Topics	/cmd_vel	Velocity command (geometry_msgs/Twist).
Topics	/odom	Robot pose estimate from encoder odometry (nav_msgs/Odometry).
Launch File	line_follow.launch.py	Launches all nodes; sets controller gains as ROS parameters.

4.5 Kinematics Requirements

- **Differential-drive kinematic model:** Derive the relationship between left and right wheel angular velocities (ω_L , ω_R), wheel radius (r), and wheelbase (L) to obtain the robot's linear velocity (v) and angular velocity (ω).
- **Forward kinematics (odometry integration):** Integrate wheel encoder data to estimate the robot's pose (x , y , θ) over time using the unicycle model. Plot the estimated trajectory for at least one full lap.
- **Controller design:** Implement at minimum a proportional (P) controller for lateral error. Students are encouraged to implement a full PID controller and tune gains experimentally. Document the controller equation and the effect of each gain constant.
- **Performance metrics:** Report lap time, maximum lateral deviation from the line center, and

cross-track RMS error computed from encoder odometry.

4.6 Demonstration Scenario

The robot is placed at a designated start point on a closed-loop track printed or taped on a flat surface. The track must include a minimum of **two 90-degree turns** and one straight segment of at least 80 cm. The robot must complete **two consecutive laps** autonomously without human intervention, maintaining contact with the line at all times. Lap time must be measured and reported.

4.7 Deliverables

- Technical report including kinematic model derivation, controller design, sensor calibration procedure, and performance analysis.
 - ROS 2 package with `line_follow.launch.py` and documented `README.md`.
 - ROS 2 bag file containing at least two consecutive laps of sensor, odometry, and command data.
 - Trajectory plot: estimated robot path (x, y) from encoder odometry overlaid on the actual track layout.
 - Video recording (3–5 minutes) showing two autonomous laps and narrated explanation of control logic.
 - PID/P gain tuning log: table of tested gains and resulting performance metric for each trial.
-

5. Project C — Landmark-Based Navigation Robot

Project C

Landmark-Based Navigation Robot (Land or Aquatic)

Design a robot that autonomously navigates between waypoints by detecting and localizing against visible landmarks or sea marks, with pose estimation grounded in geometric kinematics.

5.1 Background and Motivation

Landmark-based navigation is a key capability for both terrestrial and marine autonomous systems. Instead of relying on GPS or pre-built maps, the robot perceives distinguishable environmental features — visual markers, buoys, colored beacons, or ArUco tags — and uses geometric relationships to localize itself and plan motion toward target waypoints. This project is applicable to ground robots navigating by visible markers *or* to surface/underwater aquatic robots navigating by sea marks (buoys, channel markers, or pier structures).

Students may choose either a **wheeled/tracked ground robot** or an **aquatic surface robot (boat/USV)** as their platform. Both variants share the same kinematic modeling approach and ROS architecture requirements; they differ only in their actuation system and the physical environment of the demonstration.

5.2 Sensor Requirements

- **Primary perception sensor:** A monocular or stereo camera capable of detecting and identifying at least **three distinct landmarks** in the environment. Landmarks may be:
 - ArUco markers (recommended for ease of ID and pose estimation)
 - Colored geometric shapes (e.g., red cone, yellow buoy silhouette)
 - QR codes or custom fiducial markers
- **Heading sensor:** A digital compass (magnetometer) or IMU to provide absolute or relative heading. Required for bearing computation between robot and landmarks.
- **Range sensor (optional but recommended):** Ultrasonic or LiDAR ranging for distance estimation as a redundant localization cue.
- All sensor outputs must be published to ROS 2 topics. Camera feed must be published on `/camera/image_raw`.

5.3 Actuation Requirements

Ground Variant

- Differential-drive or Ackermann-steered wheeled robot with encoder feedback on drive wheels.
- Commands expressed as `geometry_msgs/Twist` on `/cmd_vel`.

Aquatic Variant (Surface/USV)

- Two or more thrusters (e.g., bilge pump motors or brushless DC motors with propellers) providing surge (forward/backward) and yaw (turning) control.
- Thruster commands may be expressed as differential thrust: a left thrust value and a right thrust value published as `std_msgs/Float32MultiArray` on `/thruster_cmd`.
- Aquatic platforms must include a waterproofing plan in the technical report (electronics

enclosure, cable management).

5.4 ROS Architecture Requirements

Nodes	landmark_detector	Processes camera image; publishes detected landmark IDs and pixel positions on /landmarks.
Nodes	pose_estimator	Fuses landmark bearing/distance data with heading sensor; publishes estimated robot pose on /robot_pose.
Nodes	waypoint_navigator	Accepts a list of waypoints; computes heading and speed commands to reach each in sequence.
Nodes	motion_controller	Converts navigator output to /cmd_vel or /thruster_cmd.
Topics	/camera/image_raw	Raw camera feed (sensor_msgs/Image).
Topics	/landmarks	Detected landmark list with IDs and bearing (custom msg or vision_msgs/Detection2DArray).
Topics	/robot_pose	Estimated 2D pose: x, y, theta (geometry_msgs/Pose2D).
Services	/set_waypoints	Accepts a list of Cartesian waypoints; triggers navigation sequence.
Launch File	landmark_nav.launch.py	Launches all nodes; sets landmark map parameters as YAML config.

5.5 Kinematics Requirements

- **Platform kinematic model:** Derive the forward kinematic model for the chosen platform (unicycle model for differential-drive ground robot; surge/yaw model for USV). State and justify all assumptions (e.g., non-holonomic constraints, no slip).
- **Landmark geometry and triangulation:** Given known landmark positions in the environment coordinate frame and measured bearing angles from the robot's camera, derive the geometric triangulation equations used to estimate robot position (x, y). Two approaches are acceptable: bearing-only triangulation (using at least two landmarks) or bearing-and-range (using at least one landmark).
- **Bearing computation:** Show the relationship between pixel coordinates of a detected landmark in the camera image and the bearing angle ϕ relative to the robot's heading, using the camera's field of view (FOV) and image resolution. Calibration of intrinsic camera parameters is required.
- **Waypoint heading control:** Derive the pure-pursuit or proportional heading controller used to steer the robot toward each waypoint given the estimated pose and waypoint coordinates.

5.6 Demonstration Scenario

Three or more landmarks are placed at **known positions** within a defined arena (indoor or outdoor). The robot starts at a known or estimated initial pose and must navigate autonomously to **three designated waypoints**, using landmark detection for localization at each waypoint. The order of waypoints must be specified in a YAML configuration file loaded at launch. The robot must arrive within **20 cm of each waypoint** (measured from ground truth). For the aquatic variant, the demonstration may be conducted in a pool, lake, or large container.

5.7 Deliverables

- Technical report including kinematic model derivation, triangulation geometry, camera calibration results, and localization accuracy analysis.
 - ROS 2 package with `landmark_nav.launch.py`, landmark map YAML, and documented README.md.
 - ROS 2 bag file containing a complete three-waypoint navigation run (camera feed, pose estimates, and commands).
 - Localization accuracy table: estimated vs. true robot position at each waypoint (error in cm).
 - Camera calibration report: intrinsic matrix K, distortion coefficients, and calibration image set.
 - Video recording (3–5 minutes) showing full autonomous three-waypoint navigation with landmarks clearly visible, and narrated explanation of localization logic.
-

6. Unified Marking Rubric (All Projects)

Each project is marked out of **100 points**, distributed across the five assessment categories below. The same rubric applies to all three projects; project-specific performance criteria are defined within each project section above.

1	Sensing Implementation	20	Excellent (18–20): All required sensors integrated; data published at correct rates; calibration documented; signal quality demonstrated. Satisfactory (12–17): Core sensors functional; minor calibration gaps or occasional dropped data. Insufficient (0–11): Missing required sensors or persistent sensor failures during demo.
2	Actuation and Control	20	Excellent (18–20): Smooth, accurate actuation; velocity/acceleration profiles applied; e-stop functional. Satisfactory (12–17): Core actuation functional with minor oscillation, overshoot, or tuning gaps. Insufficient (0–11): Actuators fail during demo or safety requirements not met.
3	ROS Architecture	25	Excellent (22–25): All required nodes/topics/services present; launch file works; bag file complete; README sufficient for reproduction. Satisfactory (15–21): Main nodes functional; minor missing topics or

			incomplete launch file. Insufficient (0–14): ROS architecture incomplete; system cannot be launched from repository.
4	Kinematics	25	Excellent (22–25): Rigorous derivation; correct implementation; validated against measurements; singularity / workspace analysis included. Satisfactory (15–21): Correct model with implementation gaps or partial validation. Insufficient (0–14): Kinematic model incorrect, missing, or not connected to the running system.
5	Technical Report Quality	10	Excellent (9–10): Clear structure; complete sections; all figures and tables labeled; experimental results interpreted; professional engineering writing. Satisfactory (6–8): Readable report with minor omissions or unclear figures. Insufficient (0–5): Major sections missing or results not discussed.

Note on the demonstration: The live demonstration (or submitted video) is the primary evidence for Categories 1–3. Rubric scores for these categories will be capped at the *Satisfactory* band if no working demonstration is provided, regardless of report quality. Category 4 (Kinematics) and Category 5 (Report) may be assessed from the written report alone.

7. Technical Report Format and Submission Instructions

7.1 Required Report Structure

- 1. Title Page** — Project title, project letter (A/B/C), student name(s) and ID(s), course name, submission date.
- 2. Abstract** — 200–250 words summarizing the system built, methods used, and key results.
- 3. System Description** — Hardware platform, component list with specifications, block diagram of the full system.
- 4. Sensing** — Sensor selection rationale, calibration procedure, noise characteristics, ROS topic specifications.
- 5. Actuation** — Motor selection rationale, driver circuit schematic, velocity/force profiles, e-stop design.
- 6. ROS Architecture** — Node graph (rqt_graph screenshot or hand-drawn), topic and service list with message types, launch file description.
- 7. Kinematics** — Full mathematical derivation (DH table for Project A; unicycle model for Projects B and C; triangulation for Project C). All symbols defined. Derivations step-by-step.
- 8. Experimental Results** — Quantitative results against the project's performance metrics. Plots, tables, and error analysis as specified in the project deliverables section.
- 9. Discussion** — Comparison of achieved vs. expected performance; explanation of error sources; proposed improvements.
- 10. Conclusion** — Summary of what was achieved and learned.
- 11. References** — IEEE format, minimum 5 references.
- 12. Appendix** — Full code listings (or link to Git repository), additional plots, CAD drawings.

7.2 Formatting Guidelines

- Font: Times New Roman 12pt (body); Arial 10pt (figure captions and table headers).
- Margins: 2.5 cm all sides. Single-column layout. A4 paper size.
- Equations: numbered sequentially with right-aligned equation numbers in parentheses.
- Figures: numbered sequentially, captioned below the figure. Minimum resolution 150 DPI.
- Tables: numbered sequentially, captioned above the table.
- Page limit: minimum 15 pages, maximum 35 pages (excluding appendices).
- File format: PDF, generated from the source document.

7.3 Code Repository Requirements

- Hosted on GitHub, GitLab, or equivalent platform. Repository must be made accessible to the instructor (private repo with access granted, or public repo).

- Must include: README.md with setup and launch instructions; requirements.txt or package.xml with all dependencies; ROS 2 package structure (src/, launch/, config/); and ROS 2 bag files (or download link if file size exceeds 100 MB).
- Commit history must show progressive development (not a single final commit). Regular commits are evidence of original work.

7.4 Submission Deadline and Method

All deliverables (PDF report + code repository link + ROS bag file + video link) must be submitted via the course Learning Management System (LMS) portal by the deadline posted in the course calendar. Late submissions will be penalized 10 points per 24-hour period. No submissions will be accepted more than 5 calendar days after the deadline.

8. Academic Integrity Policy

All work submitted must be the original work of the submitting student(s). The following guidelines apply:

- Use of open-source libraries (e.g., OpenCV, ROS standard packages, MoveIt) is permitted and encouraged. All third-party libraries must be cited in the References section.
 - Code generated with the assistance of Large Language Model (LLM) tools (e.g., GitHub Copilot, ChatGPT) must be disclosed in the report's Appendix, with a description of which sections were AI-assisted and how the output was reviewed, tested, and integrated. AI-assisted code that is not disclosed will be treated as academic misconduct.
 - Datasets, CAD models, or hardware schematics taken from public sources must be cited. Students may adapt but not copy without attribution.
 - Group projects (up to 3 students) must include a **contribution statement** clearly describing each member's specific contributions. All group members are equally responsible for the integrity of the entire submission.
-